

COMPSCI 340

Tutorial 1

xv6

- Modern reimplementation of Unix v6
- Designed for teaching Operating Systems – minimal & readable
- Runs on RISC-V architecture (via QEMU)
- Supports: processes, virtual memory, files, pipes, signals
- Originally written in C – roughly 6,000 lines of code
- Starting this semester, we will also be using a Rust port of xv6

UNIX v6

- Written by Ken Thompson & Dennis Ritchie, and others in C (by the guy who invented C and the guy who invented its direct predecessor)
- Designed for minicomputers & mainframes
- The ancestor of macOS & Linux



C Language

- Sequel to B by Ken Thompson
- Developed alongside Unix at AT&T's Bell Labs by Dennis Ritchie & Brian Kernighan in the 70s
- Low level and very simple language
 - Pointers allow direct access to memory
 - C code maps closely to generated assembly code
- Highly portable
 - Same code can be compiled for different platforms
 - C compilers exist for almost every architecture
- C is widely used in OS development
 - Linux
 - Windows
 - macOS

Rust Language

- Written by Graydon Hoare at Mozilla in 2006
- Low-level but complex language
 - Focuses on type safety, memory safety, encapsulation, and concurrency
 - The Rust compiler performs a lot of work to reduce the overhead of memory safety
 - Rust is generally as fast as C code at runtime
- Makes use of a “borrow checker” to eliminate memory safety bugs
 - The compiler checks and keeps track of all the variables and where they are being used
 - This eliminates memory leaks, use after free, and other memory-based bugs

Assignment 1

- Intro to xv6
- You can choose either language, but you can't switch halfway
- To make sure the development is simple, we will be using a VM
 - Ubuntu 24.04 LTS
 - Oracle VirtualBox is suggested, you may use FlexIT
- Marking will be happening in VirtualBox

Assignment 1 - Versions

You can choose to complete the assignment in either C or Rust.

- There is no difference in what you will be implementing
- You will need to complete the whole assignment in one language
- You can switch languages for different assignments
- It is up to you which language you choose
 - Neither language is easier than the other

Assignment 1 - Tips

- Get your development environment set up early
- Take a look at the existing programs
 - The entire OS is small enough to read through
- You can't use the standard library in either language
 - You will need to use the functions provided by the xv6 kernel
 - With Rust you will need to add `#![no_std]` and `#![no_main]`
- If you're stuck – Ask on Ed Discussion

Getting Started in As1

Demo time!

- 1) Set up a Unix development environment
- 2) Install the required tools
- 3) Clone the appropriate git repository
- 4) Follow the instructions to compile and run xv6 via QEMU

Rust Basics

- There is a book <https://doc.rust-lang.org/stable/book/>
 - If you're serious, read chapters 1-6 *at least*, I recommend 9 & 12
 - You can learn as you go, but you will make more mistakes
- Everything is done using cargo – like npm or uv
 - `cargo run` / `cargo build`
- There is a cargo book
- Cargo is installed using rustup
- There is a rustup book
- Rust people like books

Rust Basics

- Variables: `let name: type = value;`
 - Variables are non-mutable by default
 - `let mut name: type = value;`
- Types: `i8 i16 i32 i64 f32 f64 bool char`
- Special Types: `Enum (Error & Option) Vec< , > [ ]`
- Data structures: `struct name {field: type, ...}`
- Functions: `fn name(param: type) → return_type {}`
- Macros: `println!();`